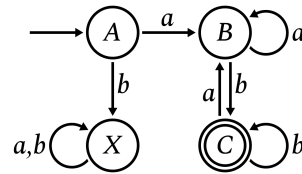


Part I

Problem 1

Language description: strings over $\Sigma = \{a, b\}$ that start with a and end with b .

One possible DFA to accept the aforementioned language is displayed below:



The formal definition of this DFA is:

DFA = $(Q, \Sigma, q_0, F, \delta)$, where

$Q = \{A, B, C, X\}$;

$\Sigma = \{a, b\}$;

$q_0 = A$;

$F = \{C\}$; and

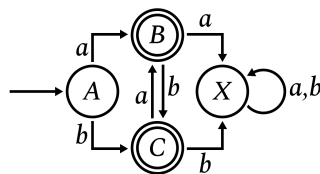
δ = state transition mapping defined in the table below:

	A	B	C	X
a	B	B	B	X
b	X	C	C	X

Problem 2

Language description: strings over $\Sigma = \{a, b\}$ that alternate between a and b .

One possible DFA to accept the aforementioned language is displayed below:



The formal definition of this DFA is:

$\text{DFA} = (Q, \Sigma, q_0, F, \delta)$, where

$Q = \{A, B, C, X\}$;

$\Sigma = \{a, b\}$;

$q_0 = A$;

$F = \{B, C\}$; and

δ = state transition mapping defined in the table below:

	A	B	C	X
a	B	X	B	X
b	C	C	X	X

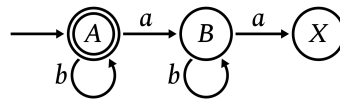
Part II

Problem 1

Language description: strings over $\Sigma = \{a, b\}$ that do not contain the substring aa .

The following is the proposed state machine:

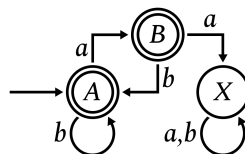
State 1 is the start state and the accepting state. It transitions to State 2 on input a and to itself on input b . State 2 transitions to itself on input b and is a trap state on input a .



The following corrections need to be made:

- State 2 needs to be marked as an accepting state;
- For State 2, input b needs to transition to State 1; and
- The trap state needs to have its inputs defined (to transition to itself).

The corrected DFA is shown below:



The formal definition of this DFA is:

$\text{DFA} = (Q, \Sigma, q_0, F, \delta)$, where

$Q = \{A, B, X\}$;

$\Sigma = \{a, b\}$;

$q_0 = A$;

$F = \{A, B\}$; and

δ = state transition mapping defined in the table below:

	A	B	X
a	B	X	X
b	A	A	X

As a test, the sample string *abbaba* will be used:

START $\rightarrow A \xrightarrow{a} B \xrightarrow{b} A \xrightarrow{b} A \xrightarrow{a} B \xrightarrow{b} A \xrightarrow{a} B \rightarrow$ **ACCEPTED**

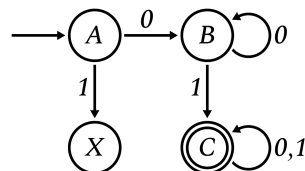
Informally, this language is defined as containing all the strings with only *a*'s and *b*'s, with the restriction that we cannot have consecutive *a*'s.

Problem 2

Language description: strings over $\Sigma = \{0, 1\}$ that start with 0 and have at least two 1's.

The following is the proposed state machine:

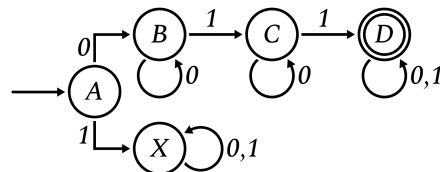
State 1 is the start state. It transitions to State 2 on input 0 and is a trap state on input 1. State 2 transitions to State 3 on input 1 and to itself on input 0. State 3 is the accepting state and transitions to itself on both inputs.



The following correction needs to be made:

- There needs to be a state identical to State 2 in-between State 2 and State 3; and
- The trap state needs to have its inputs defined (to transition to itself).

The corrected DFA is shown below:



The formal definition of this DFA is:

$\text{DFA} = (Q, \Sigma, q_0, F, \delta)$, where

$Q = \{A, B, C, D, X\}$;

$\Sigma = \{0, 1\}$;

$q_0 = A$;

$F = \{D\}$; and

δ = state transition mapping defined in the table below:

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>X</i>
0	<i>B</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>X</i>
1	<i>X</i>	<i>C</i>	<i>D</i>	<i>D</i>	<i>X</i>

As a test, the sample string 0010011 will be used:

START $\longrightarrow A \xrightarrow{0} B \xrightarrow{0} B \xrightarrow{1} C \xrightarrow{0} C \xrightarrow{0} C \xrightarrow{1} D \xrightarrow{1} D \longrightarrow \text{ACCEPTED}$

Informally, this language is defined as containing all the binary strings starting with 0 that have at least two 1's.